

[VN] So sánh monolith vs microservices. Khi nào KHÔNG nên dùng microservices?

[EN] Compare monolith vs microservices. When should microservices NOT be used?

[VN] Monolith là kiến trúc mà toàn bộ hệ thống (API, business logic, database access) được build và deploy như một khối duy nhất. Nó đơn giản, dễ phát triển ban đầu, dễ debug và phù hợp với team nhỏ hoặc sản phẩm mới. Tuy nhiên, khi hệ thống lớn dần, việc scale chỉ có thể scale toàn bộ ứng dụng, khó tách biệt module và deployment chậm hơn.

Microservices là kiến trúc chia hệ thống thành nhiều service nhỏ, mỗi service độc lập, có thể deploy và scale riêng. Nó giúp hệ thống linh hoạt, dễ mở rộng, phù hợp với hệ thống lớn và nhiều team cùng phát triển. Tuy nhiên, microservices phức tạp hơn, cần quản lý network, monitoring, logging, CI/CD, và xử lý vấn đề distributed systems như latency, consistency, fault tolerance.

Khi KHÔNG nên dùng Microservices:

Không nên dùng microservices khi team nhỏ, sản phẩm còn ở giai đoạn MVP, domain chưa rõ ràng, hoặc chưa có đủ kinh nghiệm về distributed systems và DevOps. Nếu traffic thấp và hệ thống chưa cần scale độc lập, monolith sẽ đơn giản, tiết kiệm chi phí và dễ maintain hơn. Microservices chỉ phù hợp khi hệ thống đủ lớn và có nhu cầu scale, phân tách team, hoặc yêu cầu high availability rõ ràng.

[EN] Monolith is an architecture where the whole system (API, business logic, database access) is built and deployed as a single unit. It is simple, easy to start, easy to debug, and suitable for small teams or new products. However, when the system grows, scaling must be done for the whole application, modules are harder to isolate, and deployment becomes slower. Microservices split the system into small independent services, each service can be deployed and scaled separately. It provides flexibility, better scalability, and fits large systems with multiple teams. However, it adds complexity, requires strong DevOps, monitoring, networking, and needs to handle distributed system problems like latency, consistency, and fault tolerance.

When NOT to use Microservices:

You should not use microservices when the team is small, the product is still in MVP stage, the domain is not stable, or the team lacks experience in distributed systems and DevOps. If traffic is low and independent scaling is not required, a monolith is simpler, cheaper, and easier to maintain. Microservices are suitable only when the system is large enough, needs independent scaling, team separation, or strong high availability requirements.

[VN] Trong project Lodging Transport (event-driven), bạn sẽ thiết kế communication giữa các service như thế nào?

[EN] In the Lodging Transport project (event-driven), how would you design communication between the services?

[VN] Trong project Lodging Transport sử dụng event-driven pattern và gRPC, tôi thiết kế communication theo hai hướng: synchronous và asynchronous. Với các request cần phản hồi ngay như kiểm tra phòng trống hoặc lấy thông tin chi tiết, tôi dùng gRPC để các service gọi trực tiếp nhau vì gRPC nhanh, sử dụng HTTP/2 và protobuf nên hiệu năng cao và contract rõ ràng giữa các service. Với các xử lý không cần phản hồi ngay như gửi email, đồng bộ dữ liệu hoặc cập nhật trạng thái, tôi dùng event-driven thông qua SNS và SQS. Mỗi service sẽ publish event khi có thay đổi quan trọng như BookingCreated hoặc BookingConfirmed, và các service khác sẽ subscribe để xử lý độc lập. Tôi đảm bảo mỗi event có idempotency key để tránh xử lý trùng lặp, dùng schema versioning để tránh breaking change và áp dụng cơ chế retry kết hợp dead letter queue để tăng độ tin cậy. Cách thiết kế này giúp hệ thống loose coupling, dễ mở rộng và chịu lỗi tốt hơn.

[EN] In the Lodging Transport project using event-driven pattern and gRPC, I design communication in two ways: synchronous and asynchronous. For requests that require immediate response, such as checking room availability or getting booking details, I use gRPC for direct service-to-service calls because gRPC is fast, uses HTTP/2 and protobuf, and provides a clear contract between services. For processes that do not need immediate response, such as sending emails, data synchronization, or updating status, I use event-driven communication with SNS and SQS. Each service publishes events when important changes occur, such as BookingCreated or BookingConfirmed, and other services subscribe to handle them independently. I ensure each event has an idempotency key to prevent duplicate processing, apply schema versioning to avoid breaking changes, and use retry mechanisms with a dead letter queue to improve reliability. This design keeps services loosely coupled, scalable, and more fault tolerant.

[VN] Phân biệt synchronous (REST) vs asynchronous (message queue). Trade-off?

[EN] Differentiate synchronous (REST) vs asynchronous (message queue). What are the trade-offs?

[VN] Synchronous (REST) là cách giao tiếp mà service gửi request và phải chờ response ngay lập tức. Nó đơn giản, dễ hiểu, dễ debug và phù hợp với các chức năng cần kết quả ngay như kiểm tra dữ liệu hoặc xác thực. Tuy nhiên, nhược điểm là tạo sự phụ thuộc chặt giữa các service, nếu service downstream chậm hoặc lỗi thì service gọi cũng bị ảnh hưởng và có thể gây timeout. Asynchronous (message queue) là cách giao tiếp mà service gửi message vào queue và không chờ kết quả ngay. Service nhận sẽ xử lý sau. Cách này giúp loose coupling, tăng khả năng chịu lỗi và scale tốt hơn, đặc biệt với các tác vụ nền như gửi email hoặc xử lý batch. Tuy nhiên, nó phức tạp hơn, khó debug hơn và phải xử lý thêm vấn đề như retry, duplicate message và eventual consistency. Trade-off chính là synchronous đơn giản nhưng phụ thuộc cao và khó scale lớn, còn asynchronous phức tạp hơn nhưng linh hoạt, chịu lỗi và mở rộng tốt hơn.

[EN] Synchronous (REST) communication means a service sends a request and waits for an immediate response. It is simple, easy to understand, easy to debug, and suitable for operations that need instant results such as validation or data retrieval. However, it creates tight coupling between services, and if the downstream service is slow or fails, the caller is also affected and may face timeouts. Asynchronous (message queue) communication means a service sends a message to a queue and does not wait for an immediate response. The consumer processes it later. This approach provides loose coupling, better fault tolerance, and better scalability, especially for background tasks like sending emails or batch processing. However, it is more complex, harder to debug, and requires handling issues like retries, duplicate messages, and eventual consistency. The main trade-off is that synchronous is simpler but tightly coupled and less scalable, while asynchronous is more complex but more flexible, resilient, and scalable.

[VN] Làm sao để tránh tight coupling giữa các microservices?

[EN] How can you avoid tight coupling between microservices?

[VN] Để tránh tight coupling giữa các microservices, tôi tập trung vào việc giảm sự phụ thuộc trực tiếp giữa các service. Thứ nhất, ưu tiên dùng event-driven communication thay vì gọi REST đồng bộ quá nhiều, để các service chỉ quan tâm đến event chứ không phụ thuộc logic nội bộ của nhau. Thứ hai, định nghĩa API contract rõ ràng và versioning để tránh breaking change. Thứ ba, mỗi service có database riêng, không truy cập trực tiếp database của service khác. Thứ tư, áp dụng idempotency, retry và circuit breaker để giảm ảnh hưởng khi một service bị lỗi. Cuối cùng, chia domain rõ ràng theo business capability để mỗi service có trách nhiệm độc lập. Cách này giúp hệ thống dễ scale, dễ thay đổi và ít ảnh hưởng dây chuyền khi có sự cố.

[EN] To avoid tight coupling between microservices, I focus on reducing direct dependency between services. First, I prefer event-driven communication instead of too many synchronous REST calls, so services react to events without knowing internal logic of others. Second, I define clear API contracts and apply versioning to prevent breaking changes. Third, each service has its own database and does not access another service's database directly. Fourth, I use idempotency, retry mechanisms, and circuit breaker to reduce impact when a service fails. Finally, I split services clearly based on business capability so each service has independent responsibility. This approach makes the system more scalable, easier to change, and reduces chain failures.

[VN] Thiết kế API Gateway cho hệ thống banking multi-core.

[EN] Design an API Gateway for a multi-core banking system.

[VN] Trong hệ thống banking multi-core, tôi thiết kế API Gateway như một điểm vào duy nhất cho tất cả client và hệ thống bên ngoài. Gateway chịu trách nhiệm xác thực và phân quyền (JWT hoặc OAuth2), rate limiting, logging, monitoring và routing request đến đúng core instance phía sau. Vì hệ thống multi-core có nhiều core xử lý song song, Gateway cần có routing logic dựa trên một key cố định như account ID hoặc customer ID để đảm bảo request của cùng một tài khoản luôn đi về đúng core, tránh sai lệch dữ liệu. Tôi cũng triển khai load balancing, timeout, retry và circuit breaker để tăng độ ổn định. Gateway nên stateless để có thể scale ngang dễ dàng và chạy phía sau load balancer hoặc trong Kubernetes để đảm bảo high availability.

[EN] In a banking multi-core system, I design the API Gateway as a single entry point for all clients and external systems. The gateway handles authentication and authorization (JWT or OAuth2), rate limiting, logging, monitoring, and routes requests to the correct core instance. Since the system has multiple cores running in parallel, the gateway needs routing logic based on a fixed key such as account ID or customer ID to ensure that requests from the same account always go to the same core, preventing data inconsistency. I also implement load balancing, timeout, retry, and circuit breaker to improve stability. The gateway should be stateless so it can scale horizontally and run behind a load balancer or inside Kubernetes to ensure high availability.

[VN] Thiết kế hệ thống có thể scale từ 1 instance lên 100 instance.

[EN] Design a system that can scale from 1 instance to 100 instances.

[VN] Để thiết kế hệ thống có thể scale từ 1 instance lên 100 instance, trước hết tôi đảm bảo ứng dụng là stateless, không lưu session hoặc dữ liệu tạm trong memory của một instance. Tất cả trạng thái được lưu trong database hoặc cache như Redis. Tiếp theo, tôi đặt ứng dụng phía sau load balancer để phân phối traffic đều giữa các instance. Hệ thống cần hỗ trợ horizontal scaling bằng container và Kubernetes hoặc Auto Scaling Group. Tôi cấu hình health check, readiness probe và resource limits để đảm bảo scale an toàn. Database cũng cần được tối ưu như dùng connection pool, read replica hoặc partition nếu cần. Ngoài ra, tôi thiết kế logging, monitoring và autoscaling dựa trên CPU, memory hoặc request rate để hệ thống tự động scale up khi tải cao và scale down khi tải giảm.

[EN] To design a system that can scale from 1 instance to 100 instances, I first ensure the application is stateless, meaning it does not store session or temporary data in local memory. All state is stored in a database or cache like Redis. Next, I put the application behind a load balancer to distribute traffic evenly across instances. The system must support horizontal scaling using containers and Kubernetes or Auto Scaling Group. I configure health checks, readiness probes, and resource limits to ensure safe scaling. The database must also be optimized using connection pooling, read replicas, or partitioning if

needed. In addition, I implement logging, monitoring, and autoscaling based on CPU, memory, or request rate so the system can automatically scale up during high load and scale down when traffic decreases.

[VN] Làm sao để đảm bảo backward compatibility khi thay đổi API?

[EN] How can you ensure backward compatibility when changing an API?

[VN] Để đảm bảo backward compatibility khi thay đổi API, tôi tránh sửa hoặc xóa các field cũ đang được client sử dụng. Thay vào đó, tôi chỉ thêm field mới và đảm bảo chúng là optional để không làm ảnh hưởng client cũ. Nếu cần thay đổi lớn, tôi áp dụng API versioning như /v1, /v2 để client có thể chuyển đổi dần dần. Tôi cũng duy trì contract rõ ràng bằng OpenAPI hoặc protobuf và thông báo thay đổi trước cho các team liên quan. Trong gRPC hoặc event schema, tôi không thay đổi thứ tự hoặc xóa field đã tồn tại mà chỉ đánh dấu deprecated khi cần. Cách này giúp hệ thống phát triển mà không làm gián đoạn client đang sử dụng phiên bản cũ.

[EN] To ensure backward compatibility when changing an API, I avoid modifying or removing existing fields that clients are already using. Instead, I only add new fields and make them optional so old clients are not affected. If a major change is required, I apply API versioning such as /v1 and /v2 so clients can migrate gradually. I also maintain a clear contract using OpenAPI or protobuf and communicate changes early to related teams. In gRPC or event schemas, I do not change field order or remove existing fields, but mark them as deprecated when needed. This approach allows the system to evolve without breaking existing clients.

[VN] API Gateway pattern giải quyết vấn đề gì trong microservices?

[EN] What problem does the API Gateway pattern solve in microservices?

[VN] API Gateway pattern giải quyết vấn đề khi client phải gọi nhiều microservice khác nhau. API Gateway đóng vai trò điểm vào duy nhất (single entry point) cho tất cả request từ client, sau đó định tuyến request đến các service bên trong. Nó có thể xử lý các chức năng chung như authentication, rate limiting, logging, caching, request aggregation và protocol translation. Nhờ vậy client đơn giản hơn, giảm coupling với các service nội bộ và giúp hệ thống dễ quản lý, bảo mật và mở rộng.

[EN] The API Gateway pattern solves the problem where a client needs to call many different microservices. The API Gateway acts as a single entry point for all client requests and then routes them to the appropriate internal services. It can handle common concerns such as authentication, rate limiting, logging, caching, request aggregation, and protocol translation. This makes the client simpler, reduces coupling with internal services, and helps the system be easier to manage, secure, and scale.

[VN] Strangler Fig pattern dùng khi migrate từ monolith sang microservices ra sao?

[EN] How is the Strangler Fig pattern used when migrating from a monolith to microservices?

[VN] Strangler Fig pattern là một cách migrate hệ thống từ monolith sang microservices một cách dần dần. Thay vì rewrite toàn bộ hệ thống, ta xây dựng các service mới bên ngoài và dần dần chuyển từng chức năng từ monolith sang service mới. Một layer như API gateway hoặc router sẽ quyết định request nào đi vào monolith và request nào đi vào service mới. Khi nhiều chức năng đã được chuyển sang microservices, phần monolith cũ sẽ dần bị thay thế hoàn toàn. Cách này giúp giảm rủi ro, cho phép migrate từng bước và vẫn giữ hệ thống chạy ổn định trong quá trình chuyển đổi.

[EN] The Strangler Fig pattern is a way to migrate a system from a monolith to microservices gradually. Instead of rewriting the whole system, new services are built around the monolith and features are slowly moved from the monolith to the new services. A layer such as an API gateway or router decides whether a request goes to the monolith or to the new service. As more features move to microservices, the old monolith is gradually replaced. This approach reduces risk, allows step-by-step migration, and keeps the system running during the transition.

IV. Microservices & System Design

So sánh monolith vs microservices. Khi nào KHÔNG nên dùng microservices?

Trong project Lodging Transport(event-driven), bạn sẽ thiết kế communication giữa các service như thế nào?

Phân biệt synchronous(REST) vs asynchronous(message queue). Trade-off?

Làm sao để tránh tight coupling giữa các microservices?

Thiết kế API Gateway cho hệ thống banking multi-core.

Thiết kế hệ thống có thể scale từ 1 instance lên 100 instance.

Làm sao để đảm bảo backward compatibility khi thay đổi API?

API Gateway pattern giải quyết vấn đề gì trong microservices?

Strangler Fig pattern dùng khi migrate từ monolith sang microservices ra sao?